

CS1010J Lecture #09 Unit 3

Basic OOP III

A paradigm shift



Department of Computer Science
School of Computing

Circle and TestCircle Classes

- Last week, we have written an enhanced version of the **Circle** class which is partially re-produced on the next page.
- Now let's study the user class **TestCircle** in more details. We will learn:
 - How to pass an object to a method and return an object from a method.
 - How to create and operate on an array of objects.

```
class Circle {  
  
    private int radius;      // attribute 1  
    private String colour;  // attribute 2  
  
    public Circle (int rad, String str) { // Constructor  
        radius = rad;  
        colour = str;  
    }  
  
    // Return string representation  
    public String toString() {  
        return "[" + radius + ", " + colour + "];"  
    }  
  
    // Compare two circles to see if they have identical data  
    public boolean equals(Circle another) {  
        return this.radius == another.radius &&  
            this.colour.equals(another.colour);  
    }  
    // Other code omitted for brevity  
}
```

```

import java.util.*;
class TestCircleV2 { // user class version 2

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        Circle cir1 = readCircle(scanner);
        Circle cir2 = readCircle(scanner);

        System.out.println("1st circle: " + cir1);
        System.out.println("2nd circle: " + cir2);

        if ( cir1.equals(cir2) ) {
            System.out.println("Two circles have identical data");
        }
        cir1.enlarge();
        System.out.println("1st circle: " + cir1);
    }

    public static Circle readCircle(Scanner sc) { // read a circle
        System.out.print("Enter radius: ");
        int radius = sc.nextInt();
        System.out.print("Enter colour: ");
        String colour = sc.next();
        return new Circle(radius, colour); // return a Circle object
    }
}

```

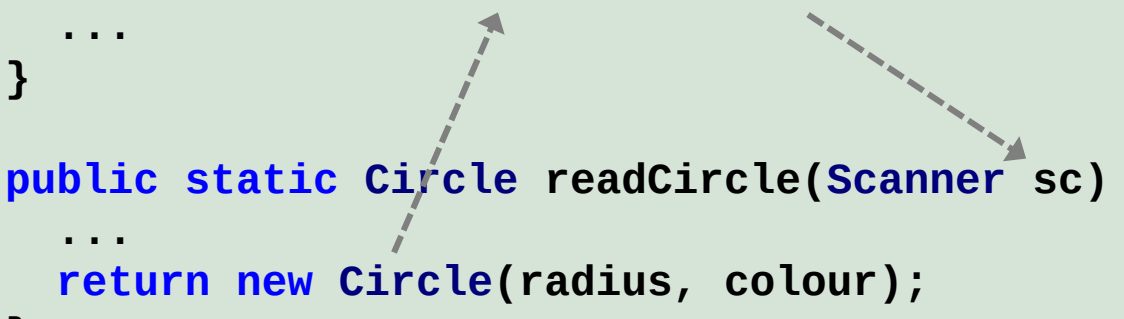
```

Enter radius: 4
Enter colour: Red
Enter radius: 4
Enter colour: Red
1st circle: [4, Red]
2nd circle: [4, Red]
Two circles have
identical data
1st circle: [8, Red]

```

Passing Objects from/to Methods (1/3)

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    Circle cir1 = readCircle(scanner);  
    ...  
}  
  
public static Circle readCircle(Scanner sc) {  
    ...  
    return new Circle(radius, colour);  
}
```



- As with **int** and **double** values, we may pass objects to methods or return objects from methods.
- When passing an object around, we are actually passing the reference (address) of that object.
 - It means **NO** duplicate of the object will be created.

Passing Objects from/to Methods (2/3)

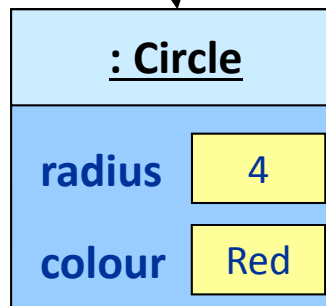
```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    Circle cir1 = readCircle(scanner);  
    ...  
}  
  
public static Circle readCircle(Scanner sc) {  
    ...  
    return new Circle(radius, colour);  
}
```

In main():



Let **cir1** point to the object.

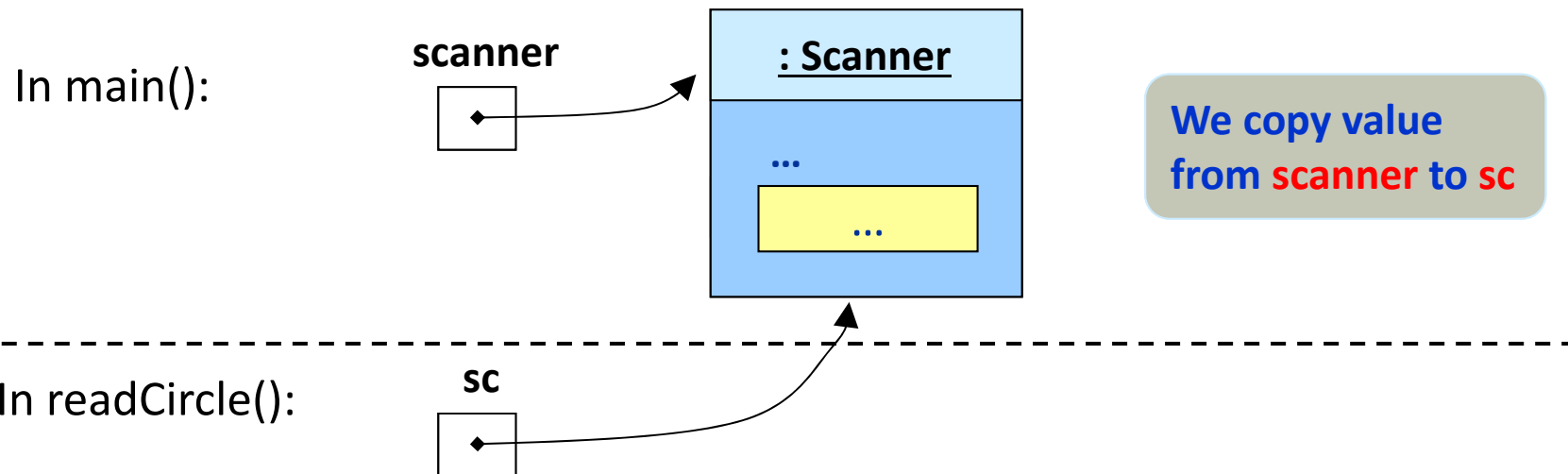
In readCircle():



Return the **reference**
(**memory address**) of the
newly created object.

Passing Objects from/to Methods (3/3)

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    Circle cir1 = readCircle(scanner);  
    ...  
}  
  
public static Circle readCircle(Scanner sc) {  
    ...  
    return new Circle(radius, colour);  
}
```



An Array of Objects

- In Java, in addition to arrays of primitive data types, we can create arrays of objects.
- Now let's create an array of **Circle** objects.

```
class TestCircleV3 { // user class version 3

    public static void main(String[] args) {

        Circle[] circles = new Circle[20];

        circles[0] = new Circle(4, "Red");
        circles[1] = new Circle(5, "White");
        ...
    }
}
```


Creating an Object Array (1/3)

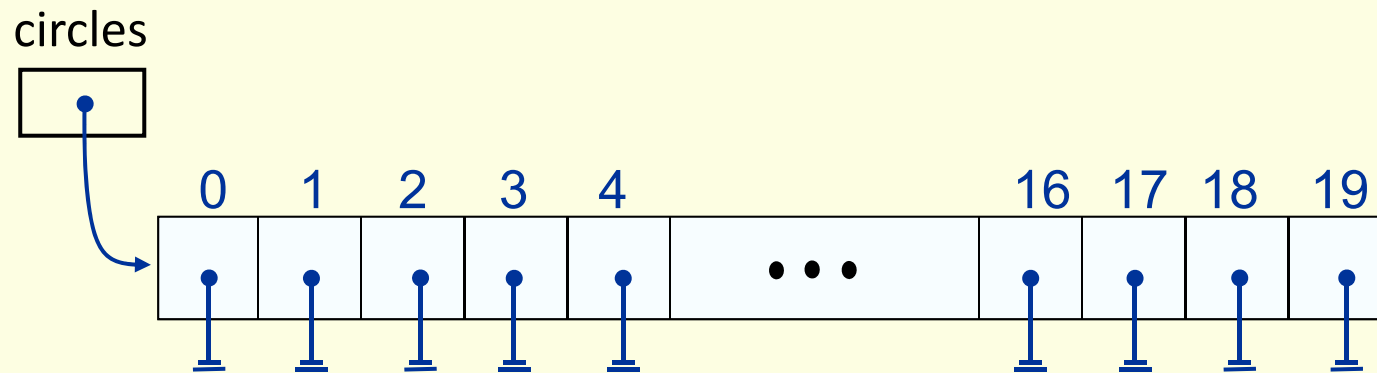
Code

A

```
Circle[] circles = new Circle[20];  
circles[0] = new Circle(4, "Red");  
circles[1] = new Circle(5, "White");
```

An array of 20 **Circle** object references is created, but the **Circle** objects themselves are not yet created.

State of
Memory



After A is executed

Creating an Object Array (2/3)

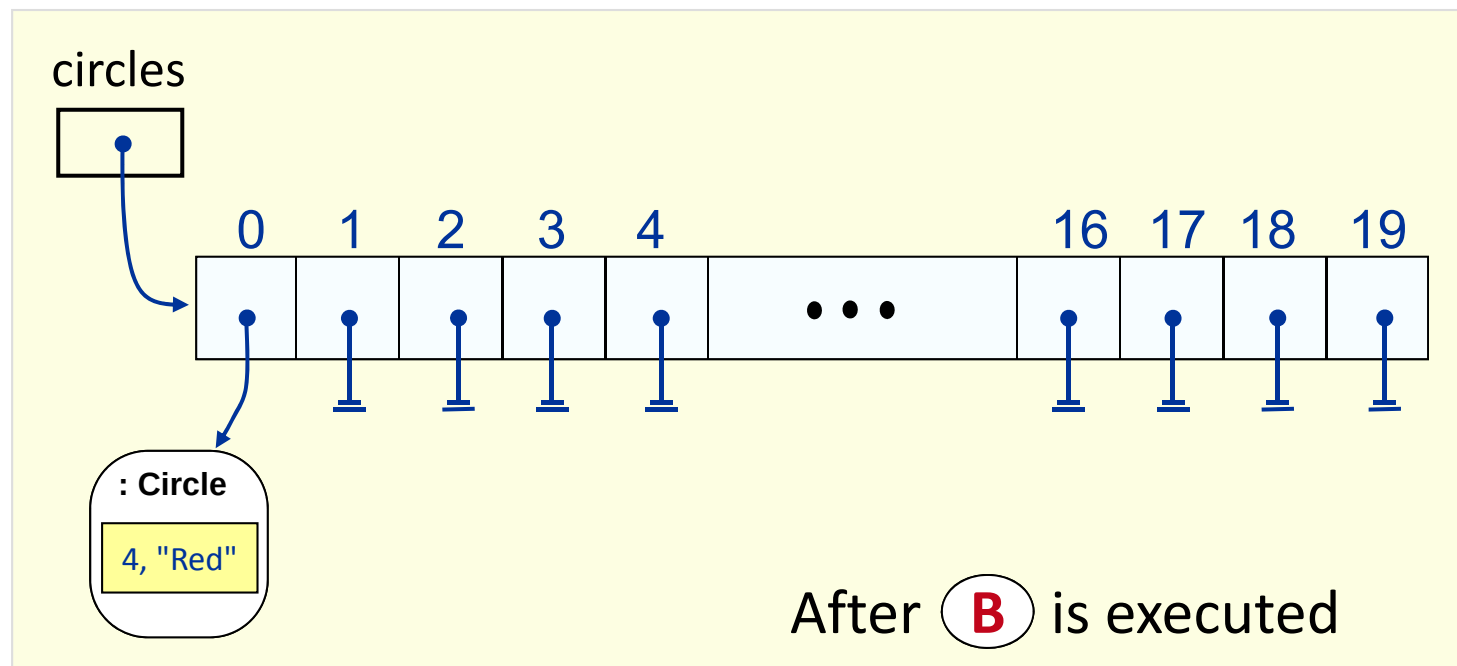
Code

B

```
Circle[] circles = new Circle[20];  
circles[0] = new Circle(4, "Red");  
circles[1] = new Circle(5, "White");
```

One **Circle** object is created and the reference to this object is placed in position 0.

State of Memory



Creating an Object Array (3/3)

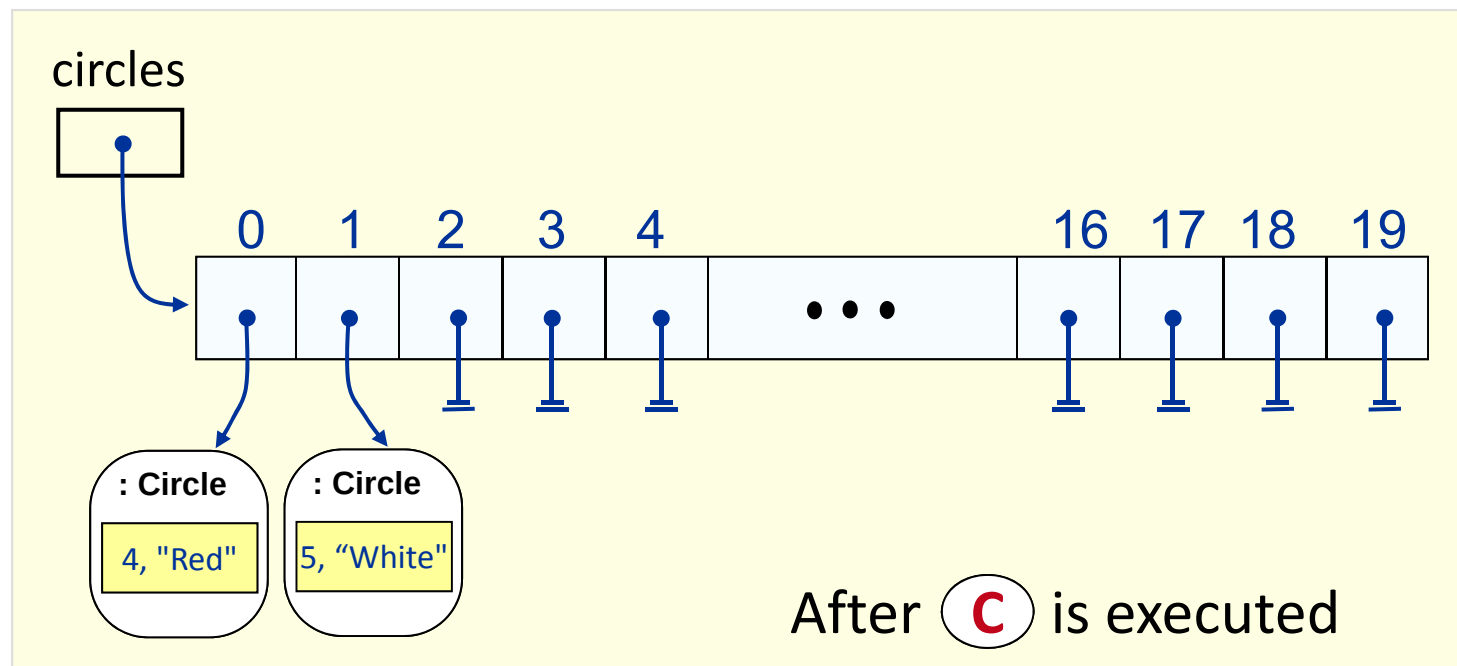
Code

Ⓒ

```
Circle[] circles = new Circle[20];  
circles[0] = new Circle(4, "Red");  
circles[1] = new Circle(5, "White");
```

Another **Circle** object is created and the reference to this object is placed in position **1**.

State of Memory



An Array of Objects

- When you create an **array of objects**, it's very common to forget to instantiate the objects.
 - Programmers often create the array itself and then think they're done – that leads to **java.lang.NullPointerException**.

```
class TestCircleV3 { // user class version 3

    public static void main(String[] args) {
        ...
        Circle[] circles = new Circle[20];
        for (int i = 0; i < circles.length; i++) {
            int radius = sc.nextInt();
            String colour = sc.next();
            circles[i] = new Circle(radius, colour);
        }
    }
}
```

Today's Summary

Basic OOP II

Basic OOP concepts and features III

- Passing object references to method calls
- Returning object references from method calls
- An array of object references

